

Advanced CUDA Python: Warp Divergence, Stencils, and Grid-Stride Scaling

Allocated Time: Exactly 60 Minutes

Mode: Solo Work Only (No pair programming, no cross-screen consultation)

Submission Medium: Public or Course-Accessible GitHub Repository
(cuda-lab-02-<your_student_id>)

Instructor: Sufyan Mustafa bin Uzayr; sufyan.mustafa@sdu.edu.kz

ZERO-INTERRUPTION OPERATING DIRECTIVE

Read this section before typing a single character.

1. **Autonomous Execution:** You are expected to operate as an independent systems engineer. The instructor will **not** assist with syntax errors, Numba compilation tracebacks, CUDA runtime errors, out-of-memory errors, Colab connection drops, or Git authorization problems.
2. **Deterministic Debugging:**
 - If you receive `CUDA_ERROR_ILLEGAL_ADDRESS`, your thread indexing is accessing unallocated VRAM. Check your boundary guards.
 - If your cell hangs indefinitely, you created an infinite loop or mismatched thread barriers. Restart the runtime (Runtime -> Restart runtime) and fix your loop indices.
 - If your output fails numerical equality checks, inspect your boundary conditions and math equations.
3. **No Paper / Digital-Only Submission:** Do not ask for paper. Everything must be implemented, benchmarked, committed, and pushed directly to GitHub.
4. **Silence Policy:** No collaboration, whispering, or screen sharing. Violations will result in an immediate zero on the lab grade.

1. REPOSITORY SPECIFICATION & STRUCTURE

Before writing kernel code, create a new Git repository named:

cuda-lab-02-<your_student_id>

Your final committed repository must match this file tree:

cuda-lab-02-<your_student_id>/

```
|— README.md           <-- Telemetry, benchmarks, answers, and integrity token
|— task1_divergence.py  <-- Warp divergence microbenchmark
|— task2_stencil_1d.py  <-- 1D Stencil kernel with boundary clamping
|— task3_grid_stride.py <-- Grid-stride loop processing arbitrary vector sizes
```

- └─ task4_sobel_2d.py <-- 2D spatial image convolution (Sobel-X)
- └─ verify_submission.py <-- Automated self-check script provided below

2. LAB TASKS

Task 1: Warp Divergence Microbenchmark (task1_divergence.py)

Core Concept: An SM schedules and executes instructions in groups of 32 threads known as **warps**. When threads within the same warp take differing computational branches (if/else), the hardware serializes execution paths, halving (or worse) theoretical throughput.

Implementation Requirements

1. Allocate an array of $N = 2^{20} = 1,048,576$ elements (np.float32).
2. Write three CUDA kernels executing 1,000 iterations per element:
 - **Kernel A (Uniform Path):** Every thread executes the identical arithmetic:

$$y[i] = y[i] \times 1.0001 + 0.0001$$

- **Kernel B (Full Divergence - Interleaved):**
 - If $\text{idx} \% 2 == 0$: execute 1,000 multiply-accumulate operations.
 - Else: execute 1,000 distinct subtract-divide operations.
 - *Hardware impact:* Adjacent threads in every warp diverge.
 - **Kernel C (Warp-Aligned Branching):**
 - Use $\text{warp_id} = \text{idx} // 32$.
 - If $\text{warp_id} \% 2 == 0$: execute Path 1.
 - Else: execute Path 2.
 - *Hardware impact:* Entire warps take Path 1 or Path 2 uniformly without intra-warp divergence.
3. Benchmark **Kernel-Only Execution Time** (exclude Host-to-Device/Device-to-Host transfers). Run a warm-up launch, then average 10 consecutive trials using `cuda.synchronize()` and `time.perf_counter()`.
 4. Log empirical values into the table in your README.md.

Task 2: 1D Boundary Stencil & Halo Protection (task2_stencil_1d.py)

Core Concept: Real-world physical simulations and DSP filters rely on neighboring elements.

Because neighboring elements at $\text{idx} = 0$ and $\text{idx} = N - 1$ do not exist, naive lookups trigger out-of-bounds read violations.

Implementation Requirements

1. Implement a 3-point smoothing filter:

$$y[i] = 0.25 \cdot x[i - 1] + 0.5 \cdot x[i] + 0.25 \cdot x[i + 1]$$

2. In `@cuda.jit def stencil_1d(d_in, d_out, N):`
 - Compute 1D global thread index: `idx = cuda.grid(1)`.
 - Enforce boundary guard: `if idx < N:`.
 - Implement boundary clamping (halo replication):
 - If `idx == 0` : read left neighbor as `d_in[0]` .
 - If `idx == N - 1` : read right neighbor as `d_in[N - 1]` .
 - Otherwise: read `d_in[idx - 1]` and `d_in[idx + 1]` .
3. Provide a host wrapper function `run_stencil(h_in)` that handles device transfers and launches blocks of 256 threads.
4. Implement an exact CPU reference validator using NumPy:


```
def cpu_stencil(arr):
    padded = np.pad(arr, (1, 1), mode='edge')
    return 0.25 * padded[:-2] + 0.5 * padded[1:-1] + 0.25 * padded[2:]
```
5. Test against an odd, non-power-of-two size: $N = 100,007$.
6. Assert `np.allclose(h_out_gpu, cpu_ref, atol=1e-4)`. Print: TASK 2 PASSED: MAX DELTA = <delta>.

Task 3: Arbitrary-Size Vector Scaling via Grid-Stride Loops (task3_grid_stride.py)

Core Concept: One-to-one thread-to-element mapping fails when data sizes exceed hardware grid limits or when memory throughput saturates. A **grid-stride loop** decouples your software problem size from the hardware thread allocation.

Implementation Requirements

1. Implement `@cuda.jit def grid_stride_scale_kernel(d_arr, factor, N):`
 - Calculate global thread index: `start = cuda.grid(1)`.
 - Calculate total grid span: `stride = cuda.gridsize(1)`.
 - Loop using stride increments:


```
for i in range(start, N, stride):
    d_arr[i] = d_arr[i] * factor
```
2. Test on a vector of size $N = 16,777,216$ (2^{24} elements).
3. **Hardware Constraint:** You are strictly forbidden from launching N threads. Configure your launch grid to:

- threads_per_block = 256
 - blocks_per_grid = 64
 - (Total hardware threads launched: $16,384 \ll 16,777,216$)
4. Provide host caller function run_grid_stride(h_arr, factor) returning the result array.
 5. Confirm on the host that all $16,777,216$ elements match factor.

Task 4: 2D Spatial Convolution - Sobel Horizontal Filter (task4_sobel_2d.py)

Core Concept: 2D grid geometries for image and spatial tensor operations require dynamic 2D block-to-grid mapping and boundary-safe 2D indexing.

The Sobel Horizontal (K_x) kernel calculates spatial derivatives along columns:

$$K_x = \begin{bmatrix} -1.0 & 0.0 & +1.0 \\ -2.0 & 0.0 & +2.0 \\ -1.0 & 0.0 & +1.0 \end{bmatrix}$$

Implementation Requirements

1. Create a 2D float32 input matrix of shape 2048×2048 .
2. Implement @cuda.jit def sobel_x_kernel(d_in, d_out, rows, cols):
 - Extract 2D coordinates: col, row = cuda.grid(2).
 - Ensure execution stays within boundaries:
 - For internal pixels ($0 < row < rows - 1$ and $0 < col < cols - 1$), compute:

$$\begin{aligned} d_out[row, col] = & -1.0 \cdot d_in[row - 1, col - 1] + 1.0 \cdot d_in[row - 1, col + 1] \\ & - 2.0 \cdot d_in[row, col - 1] + 2.0 \cdot d_in[row, col + 1] \\ & - 1.0 \cdot d_in[row + 1, col - 1] + 1.0 \cdot d_in[row + 1, col + 1] \end{aligned}$$
 - For outer border pixels ($row == 0$, $row == rows - 1$, $col == 0$, or $col == cols - 1$), set $d_out[row, col] = 0.0$.
3. Launch with 2D block dimensions: threads_2d = (16, 16). Compute grid geometry dynamically to cover 2048×2048 .

4. Provide host wrapper function `run_sobel(h_img)` returning the filtered matrix.

3. AUTOMATED VERIFICATION SUITE

Save this file as `verify_submission.py` in your repository. Run it locally in your environment. Do not submit your repository until this script passes all checks and prints your integrity token.

```
"""
```

```
AUTONOMOUS VERIFICATION & INTEGRITY TOKEN GENERATOR
```

```
Run: python verify_submission.py
```

```
"""
```

```
import sys
```

```
import hashlib
```

```
import numpy as np
```

```
from numba import cuda
```

```
def main():
```

```
    print("=" * 60)
```

```
    print("RUNNING CUDA LAB 02 AUTONOMOUS VERIFICATION")
```

```
    print("=" * 60)
```

```
    student_id = input("Enter your Student ID: ").strip()
```

```
    if not student_id:
```

```
        print("[FAIL] Student ID cannot be empty.")
```

```
        sys.exit(1)
```

```
# 1. Verify Task 2
```

```
try:
```

```
    import task2_stencil_1d as t2
```

```
    N = 10007
```

```
    test_in = np.sin(np.linspace(0, 10, N)).astype(np.float32)
```

```
    h_gpu = t2.run_stencil(test_in)
```

```
    h_cpu = t2.cpu_stencil(test_in)
```

```
    assert np.allclose(h_gpu, h_cpu, atol=1e-4), "Task 2 output mismatch against CPU"
```

```
    print("[PASS] Task 2 (1D Stencil & Clamping)")
```

```
except Exception as e:
```

```
    print(f"[FAIL] Task 2: {e}")
```

```
    sys.exit(1)
```

```
# 2. Verify Task 3
```

```
try:
```

```
    import task3_grid_stride as t3
```

```
    N = 100000
```

```

test_arr = np.ones(N, dtype=np.float32)
factor = 4.25
res = t3.run_grid_stride(test_arr, factor)
assert np.allclose(res, factor), "Task 3 elements not uniformly scaled"
print("[PASS] Task 3 (Grid-Stride Scaling)")
except Exception as e:
    print(f"[FAIL] Task 3: {e}")
    sys.exit(1)

# 3. Verify Task 4
try:
    import task4_sobel_2d as t4
    test_img = np.ones((64, 64), dtype=np.float32)
    sobel_res = t4.run_sobel(test_img)
    # Uniform image must have zero interior gradient
    assert np.max(np.abs(sobel_res[1:-1, 1:-1])) < 1e-5, "Task 4 interior gradient != 0 for flat
field"
    assert np.all(sobel_res[0, :] == 0.0), "Task 4 border rows not zeroed"
    assert np.all(sobel_res[:, 0] == 0.0), "Task 4 border columns not zeroed"
    print("[PASS] Task 4 (2D Sobel Horizontal)")
except Exception as e:
    print(f"[FAIL] Task 4: {e}")
    sys.exit(1)

# Cryptographic Checksum Token Generation
hasher = hashlib.sha256()
hasher.update(student_id.encode('utf-8'))
try:
    device_name = cuda.get_current_device().name
    hasher.update(device_name if isinstance(device_name, bytes) else
device_name.encode('utf-8'))
except Exception:
    hasher.update(b"UNKNOWN_CUDA_DEVICE")
hasher.update(h_gpu[:32].tobytes())
hasher.update(sobel_res[:8, :8].tobytes())
token = hasher.hexdigest()[:20].upper()

print("\n" + "=" * 60)
print(f"VERIFICATION SUCCESSFUL")
print(f"OFFICIAL SUBMISSION TOKEN: {token}")
print("=" * 60)
print("Copy this token directly into your README.md.\n")

```

```
if __name__ == "__main__":  
    main()
```

4. MANDATORY README.md TEMPLATE

Create your repository README.md using the exact structure below:

CUDA Lab 02: Advanced Geometries & Stencils

****Student ID:**** <Your Student ID>

****Allocated GPU Node:**** <e.g., Tesla T4 / RTX 3080>

****CUDA Compute Capability:**** <e.g., 7.5>

****Official Verification Token:**** <PASTE_TOKEN_HERE>